Managing LLM Model Capacity in Production

1. Why Capacity Management Matters

When you deploy an LLM in production, the model itself is only one part of the problem. The real challenge is handling real-world load — thousands of users sending requests at the same time, unpredictable traffic spikes, rate limits from API providers, and GPU memory constraints. If you do not manage capacity properly, you end up with slow responses, failed requests, high costs, or complete outages.

Capacity management in LLM systems means making sure you always have enough compute to serve requests on time, without wasting money on idle resources.

The Key Metrics to Track

Metric	What it means	Target to aim for
Latency (TTFT)	Time to first token — how long before the model starts replying	< 500ms for interactive apps
Throughput	Tokens generated per second across all users	Maximize without breaching latency SLA
GPU utilization	How busy your GPUs are	70-85% (higher = wasted requests, lower = wasted money)
Queue depth	How many requests are waiting to be processed	Near zero in steady state
Error rate	% of requests that fail or time out	< 0.1%

2. Scaling Strategies

Horizontal Scaling — Add More Instances

The most straightforward approach. Run multiple copies of your model and split traffic between them. When load increases, spin up more instances. When it drops, scale down.

- Works well with managed services like AWS SageMaker, Azure ML, Google Vertex AI — they handle autoscaling for you
- For self-hosted models (using vLLM, TGI, or Triton), use Kubernetes with a Horizontal Pod Autoscaler (HPA) to scale replicas based on GPU utilization or request queue length
- Key watch-out: cold start time. Spinning up a new LLM instance takes 30-120 seconds. Pre-warm instances during predictable traffic ramps.

Vertical Scaling — Bigger Hardware

Use more powerful GPUs instead of more instances. Upgrade from A10G to A100 to H100.

- Better for very large models that cannot be split across multiple small GPUs
- Diminishing returns at some point — two A100s are not always twice as good as one
- Use when a single request needs a lot of memory (very long context, large batch size)

Model Parallelism — Split One Model Across GPUs

For very large models (70B+ parameters) that do not fit on a single GPU, you split the model across multiple GPUs.

- Tensor parallelism: split individual weight matrices across GPUs. All GPUs work on each request simultaneously. Fast but requires fast GPU interconnects (NVLink).
- Pipeline parallelism: each GPU holds a different set of layers. Requests flow through GPUs sequentially like an assembly line.
- vLLM handles both automatically — just set tensor_parallel_size when launching the server

Request Batching

Instead of processing one request at a time, group multiple requests into a single batch and run them together on the GPU. This dramatically improves throughput.

- Static batching: collect N requests and process them together. Simple but adds wait time for early arrivals.
- Continuous / dynamic batching: requests join and leave the batch mid-generation. vLLM and TGI do this automatically. Much better GPU utilization.
- Rule of thumb: continuous batching alone can increase throughput by 10-23x compared to naive single-request serving

> **Key concept:** PagedAttention (used in vLLM) manages the KV cache like virtual memory — it eliminates 60-80% of GPU memory waste from fragmentation, allowing you to fit many more concurrent requests on the same GPU.

---

3. Load Balancing

Load balancing is how you distribute incoming requests across your fleet of model instances. Done right, it keeps all instances busy without overloading any of them.

Common Load Balancing Strategies

- **Round robin:** requests go to instances in rotation. Simple. Works when all requests are roughly the same size.
- **Least connections:** send each new request to the instance with the fewest active requests. Better for variable-length requests.
- **Consistent hashing:** route requests from the same user/session to the same instance. Important for KV cache reuse — the model can reuse previous conversation context and respond faster.
- **Predicted-latency routing:** estimate how long a request will take (based on prompt length) and route it to the instance best suited to handle it. Best option for mixed workloads.

Multi-Provider Load Balancing

40% of production teams now use multiple LLM providers (e.g. OpenAI + Anthropic + Azure OpenAI) to avoid vendor lock-in and stay within rate limits. A gateway layer (like LiteLLM, Portkey, or a custom proxy) sits in front and routes requests based on:

  - Which provider has available capacity right now
  - Which provider is cheapest for this type of request
  - Which provider is fastest at this moment (health-aware routing)

> **Resilience tip:** Always configure fallback providers. If your primary provider hits a rate limit or goes down, automatically retry the request on a secondary provider. This is how teams push uptime from 97% to 99.97%.

4. Rate Limiting and Throttling

Rate limits are caps that API providers impose on how many requests or tokens you can send per minute. Hitting them causes 429 errors. Managing them well is critical in production.

Types of Rate Limits to Know

- RPM (Requests Per Minute): max number of API calls per minute
- TPM (Tokens Per Minute): max input + output tokens per minute
- Concurrent request limit: max number of requests running at the same time

How to Handle Rate Limits

- **Exponential backoff with jitter:** when you hit a 429, wait and retry. Double the wait time each attempt, add random jitter to avoid thundering herd. Most SDKs do this automatically.
- **Request queuing:** put incoming requests into a queue and release them at a rate that stays within limits. Use a token bucket or leaky bucket algorithm.
- **Priority queues:** not all requests are equal. Critical user-facing requests jump the queue. Background batch jobs wait.
- **Circuit breaker:** if a provider fails repeatedly, stop sending requests to it for a period (e.g. 30 seconds) and route to backup instead. Prevents cascading failures.

User-Level Rate Limiting

For your own API, you should also rate-limit individual users to prevent one heavy user from consuming all capacity:

  - Assign per-user token budgets per day or per minute
  - Use Redis to track usage in real time across all your servers
  - Return clear error messages with retry-after headers so users know when they can try again

5. Cost Optimization

LLM inference costs money fast — mostly GPU compute and API tokens. These are the most effective ways to keep costs under control without hurting quality.

Semantic Caching

Cache LLM responses and reuse them for similar future questions. Unlike regular caching (exact string match), semantic caching matches by meaning using embeddings.

- If a user asks 'what is the capital of France?' and someone already asked 'what city is the capital of France?', return the cached answer
- Tools: GPTCache, Redis with vector similarity, LangChain's caching layer
- Can cut API costs by 30-50% for use cases with repeated or similar queries (FAQs, support bots)

Model Routing — Use Smaller Models When You Can

Not every request needs your most powerful (most expensive) model. Route simple requests to a smaller, cheaper model and only send complex ones to the big model.

Request Type	Model to Use	Why

Request Type	Model to Use	Why
Simple FAQ, short classification	GPT-4o-mini, Claude Haiku	Fast, cheap, good enough
Complex reasoning, multi-step tasks	GPT-4o, Claude Sonnet/Opus	Accuracy matters more than cost
Summarization, extraction	Mid-tier model	Balance of speed and quality
Internal batch processing	Cheapest available	No SLA pressure, cost priority

Prompt Optimization

- Shorter prompts = fewer input tokens = lower cost. Trim system prompts aggressively.
- Remove conversation history older than N turns to keep context windows lean
- Use structured outputs (JSON mode) to reduce verbose freeform responses

Batching for Async Workloads

For non-real-time jobs (generating reports, processing uploaded docs, batch embeddings), use the provider's batch API — it is typically 50% cheaper than the standard API.

---

6. Quick Reference

Problem	Solution
Traffic spike crashes the service	Autoscaling (HPA on Kubernetes), pre-warm instances
Single instance is a bottleneck	Horizontal scaling, load balancer in front
GPU memory runs out	PagedAttention (vLLM), reduce batch size, quantization
Hitting provider rate limits	Request queue, exponential backoff, multi-provider routing
Responses are too slow	Continuous batching, streaming tokens, model distillation
Inference costs too high	Semantic cache, smaller model routing, batch API, shorter prompts
Provider goes down	Circuit breaker, automatic failover to backup provider
One user drowns everyone else	Per-user rate limits, priority queues

> **Core principle:** In production LLM systems, most performance problems are infrastructure problems, not model problems. Get the serving layer right first.